



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Design Characteristics and Metrics

Francesco Guerra
francesco.guerra@unimore.it

OBJECTIVES

- ▶ Describe the characteristics of a good design.
- ▶ Understand metrics that have been devised to measure complexity of design.
- ▶ Understand metrics for object-oriented design
- ▶ Discuss cohesion attribute and program slices.
- ▶ Describe coupling attributes.

Characterizing Design

- ▶ There is no single definition of a good design. A good design is characterized by several attributes.
- ▶ **From the requirements:**
 - ▶ **Consistency across the design:**
 - ▶ Common UI (appearance and logical flow)
 - ▶ Common error processing
 - ▶ Common reports
 - ▶ Common system interfaces
 - ▶ Common help
 - ▶ All design carried to the same depth level
 - ▶ **Completeness of the design:**
 - ▶ All requirements are accounted for
 - ▶ All parts of the design are carried to completion at the same depth level

Characterizations of Design Attributes

- ▶ Early software engineering focused on **detailed design and coding-level attributes**, not architectural design.
 - ▶ Programs and modules were long considered the **most important artifacts**.
 - ▶ Measurements and metrics targeted **detailed design and coding elements**.

Principles of Software Metrics

- ▶ *“The purpose of software metrics is to make assessments throughout the software life cycle as to confirm whether the software quality requirements are being met.*
- ▶ *The use of software metrics reduces subjectivity in the assessment and control of software quality by providing a quantitative basis for making decisions about software quality.*
- ▶ *However, the use of software metrics does not eliminate the need for human judgment in software evaluations.*
- ▶ *The use of software metrics within an organization or project is expected to have a beneficial effect by making software quality more visible.”*
- ▶ -IEEE Standard 1061, 1998 (Inactivated Date: 2020-03-05)

Definition of Software Metrics

- ▶ A **software metric** is a quantitative measure of the degree to which a **software system, component, or process** possesses a defined property or characteristic.
- ▶ Metrics are applied using **measurement techniques**, producing **measured values**.
 - ▶ Example: **Lines of Code (LOC)** measures the size of a source code unit by counting lines.
- ▶ A **software quality metric** is a software metric that characterizes **quality aspects**.
 - ▶ The interpretation of measured values is **strongly empirical**.

General Types of Software Metrics

▶ **Product metrics**

- ▶ Characterize an **intermediate or final product** of the software development process.
- ▶ Examples: code size, code complexity, customer satisfaction.

▶ **Process metrics**

- ▶ Characterize **methods, techniques, and tools** used in developing, implementing, and maintaining the software system.
- ▶ Examples: effectiveness of defect removal during development, response time of bug-fix processes.

▶ **Project metrics**

- ▶ Characterize the **project** in terms of time, budget, and management.
- ▶ Examples: number of software developers, staffing pattern over the software life cycle, cost, schedule, productivity.

- ▶ **Note:** A specific software metric may belong to **multiple categories**.

Adapted from Stephen H. Kan: "Metrics and
Models in Software Quality Engineering"

Product metrics (code & modules)



Code Complexity Measures

- ▶ Derived by **static source code analysis**
- ▶ Relate to and reflect aspects of the coding quality, coding complexity, and modularization
- ▶ Examples
 - ▶ Lines-of-Code (LOC, NLOC)
 - ▶ Nesting Level (NL)
 - ▶ Cyclomatic Complexity (McCabe)
 - ▶ Halstead Complexity
 - ▶ Coupling
 - ▶ Cohesion
 - ▶ Code Clones
 - ▶ Function Points

Dynamics-Related Measures

- ▶ Analyzed based on recording system dynamics at run-time
- ▶ Relate to and reflect aspects of the system behavior
- ▶ Examples
 - ▶ Code Coverage
 - ▶ Bugs-per-Line-of-Code
 - ▶ Feature Usage

Limitations of Software Metrics

- ▶ It is difficult to define general “**normal ranges**” for measured values of individual software quality metrics.
 - ▶ Normal ranges can be **empirically determined** for a specific software project.
- ▶ Simplistic measurements may do **more harm than good**.
- ▶ Measured values suggest precision, but there is a **strong tendency for professionals to display over-optimism and over-confidence**.
- ▶ Developers may adapt their work to metrics if used as **strict quality instruments**, resulting in **opportunistic, metric-driven behavior**.

Definition of LOC Metrics

► Definition

- The *lines-of-code (LOC) metrics* is a set of software metrics that quantitatively measure source code size by counting “source code lines”.
 - What is considered to be a “source code line” depends on the specific type of LOC metric.
 - LOC metrics can be applied to all types of code units such as functions, methods, classes, packages, or files.
 - LOC convey an order of magnitude of a given code unit
 - Computed by static source code analysis
- Also measured in 1000-units, abbreviated by KLOC

LOC Applications

- ▶ LOC metrics are used
 - ▶ to determine the “**order of magnitude**” of the implementation of a complex computer-based system
 - ▶ as a statistical measurements **to estimate and predict efforts.**
 - ▶ to build higher level metrics, for example, **to estimate general cost and efforts** (e.g., for maintenance, error handling, bug fixing).
- ▶ The ratio of comment lines and source code lines (**CLOC/LOC**) reflects the degree to which **source code is documented by comments.**
- ▶ LOC metrics measure “how much”
 - ▶ source code of a system is currently (or was in the past) modified, under revision, added, or removed within a given time frame
 - ▶ LOC metrics can be used to quantitatively determine the degree to which source code changed or evolved over time.
 - ▶ individual developers contributed to a system's implementation in given time frame.

LOC Metrics

- ▶ There are different variants of LOC metrics according to the definition of “source code line”:
 - ▶ LOC: physical lines: a line corresponds to a line in the source code's text representation
 - ▶ SLOC: source code lines of code, without comment and blank lines
 - ▶ CLOC: comment lines of code
 - ▶ NCLOC: non comment lines of code; does exclude comments, headers and footers such as include statements, braces of function bodies, etc.
 - ▶ LLOC: logical lines of code, counts the number of statements by parsing the source code, regardless of their layout within a single line or across many lines
 - ▶ BLOC: blank lines: lines without text, used to format the source code
- ▶ All LOC metrics can be computed in an easy, fast, and robust way.

LOC example

```
public static int factorial(int n){  
  
    // n= number of interest  
    int result = 1;  
    for(int i = 2; i <= n; i++)  
        result *= i;  
    return result;  
}
```

- ▶ LOC = 8
- ▶ SLOC = 6
- ▶ NCLOC = 5

LOC Metrics

Disadvantages

- ▶ LOC can be correlated with effort but not with functionality, that is, it does not tell about the real complexity of an implementation.
 - ▶ For example, functions can require days to be developed but results in less than 1.000 LOC.
- ▶ LOC does not consider complexity that results from automatic code generation using metaprogramming, or frameworks.
- ▶ If LOC is used as measure of activity, developers can easily undermine that measure by “opportunistic coding styles” such as writing long-winded code.
- ▶ Implementations of complex computer-based systems frequently tend to use different languages -LOC by its nature depends on the programming language to a certain degree (way of writing comments, functions, methods, classes, etc.).
 - ▶ It is difficult to compare LOC of source code units written in different programming languages.

Code Duplication

```
1  int sum = 0;
2
3  void foo(Iterator iter){
4      for (item = first(iter); has_more(iter); item = next(iter)){
5          sum = sum + value (item);
6      }
7  }
8  int bar(Iterator iter){
9      int sum = 0;
10     for (item = first(iter); has_more(iter); item = next(iter)){
11         sum = sum + value (item);
12     }
13 }
```

- ▶ result from copying and pasting code (“code clones”)
- ▶ result from independent development activity
- ▶ are not explicitly visible or recorded
- ▶ violate the principle “Don't repeat yourself” (DRY)

Consequences of Cloning

- ▶ There are plausible arguments that code cloning increases maintenance effort.
 - ▶ Changes must be made consistently multiple times if the code is redundant.
 - ▶ Often it is not documented where code has been copied.
 - ▶ Manual search for copied code is infeasible for large systems.
 - ▶ During analysis, the same code must be read over and over again, then compared to the other code just to find out that this code has already been analyzed.
- ▶ If the code would have been implemented only once in a function, this effort could have been completely avoided.

Duplication detection

- ▶ Text-based clone search (syntactic matches)
- ▶ Token-based clone search (robust against reformatting, documentation changes, or renaming of variables, classes, or methods)
- ▶ Abstract Syntax Tree (AST) approaches (robust against differences in keywords or parentheses)
- ▶ Other more complex techniques

Clone Types

- ▶ Type-1: Identical code fragments except for variations in whitespace, layout and comments.
- ▶ Type-2: Syntactically identical fragments except for variations in identifiers, literals, types, whitespace, layout and comments.
- ▶ Type-3: Copied fragments with further modifications such as changed, added or removed statements, in addition to variations in identifiers, literals, types, whitespace, layout and comments.
- ▶ Type-4: Two or more code fragments that perform the same computation but are implemented by different syntactic variants
- ▶ There are plugins in the IDE to detect code duplications

NL Metric

- ▶ The *nesting level (NL) metric* is a software metric that quantitatively measures source code complexity by measuring the depth of enclosing of control structures one into another.
- ▶ Source codes apply nesting in several forms including
 - ▶ nested calls,
 - ▶ nested levels of parentheses in arithmetic expressions,
 - ▶ nested blocks of source code (e.g. nested if-clauses, while-clauses),
 - ▶ nested classes,
 - ▶ nested functions, etc.
- ▶ The nesting level denotes the level of containment (depth) of a logical source code block.
- ▶ Used to measure the complexity of source code units such as functions or methods.

Variations of NL Metrics

- ▶ **NL**
 - ▶ Highest nesting level of a given source code unit
- ▶ **LoopNL**
 - ▶ Highest nesting level in a loop of a given source code unit
- ▶ **NLMean**
 - ▶ Mean nesting level of multiple source code units (e.g., whole class, module)
- ▶ **NL#+**
 - ▶ Number of lines-of-code above a certain nesting level, starting at #(e.g., NL4+, NL5+)

Example

```
public String truncateAInFirst2Positions(String str) {  
    String first2Chars = str.substring(0, 2);  
    String stringMinusFirst2Chars = str.substring(2);  
    return first2Chars.replaceAll("A", "") + stringMinusFirst2Chars;  
}
```

- ▶ function with a single linear control flow, that is, no decisions nor repetitions -- $> NL_{MAX}=1$

```
public String truncateAInFirst2Positions(String str) {  
    if (str.length() <= 2)  
        return str.replaceAll("A", "");  
    String first2Chars = str.substring(0, 2);  
    String stringMinusFirst2Chars = str.substring(2);  
    return first2Chars.replaceAll("A", "") + stringMinusFirst2Chars;  
}
```

- ▶ function with an if statement (with one condition), that is, with a single decision -- $> NL_{MAX}=2$

Example (2)

```
public boolean areFirstAndLastTwoCharactersTheSame(String str) {  
    if (str.length() <= 1)  
        return false;  
    if (str.length() == 2)  
        return true;  
    String first2Chars = str.substring(0, 2);  
    String last2Chars = str.substring(str.length() - 2);  
    return first2Chars.equals(last2Chars);  
}
```

- ▶ The control flow contains two if statements (with one condition)
-- > $NL_{MAX}=2$

Example (3)

```
public void printStar() {  
    for (int i = 0; i<10; i++)  
        for (int j=0; j<10; j++)  
            if ((i+j) % 2)  
                System.out.println("*")  
            else  
                System.out.println(" ")  
}
```

- ▶ The control flow contains two nested loops and an if statement
-- > $NL_{MAX}=4$

NL properties

- ▶ High NL values generally indicate that **more effort has to spend to understand the code unit** with respect to program understanding and comprehension.
 - ▶ The underlying observation is that deeply nested code becomes extremely difficult to understand from a cognitive point of view.
- ▶ NL metric measurements can be used to **identify those parts of the source code that should be redesigned** or refactored using modularization, abstraction, or encapsulation strategies.
- ▶ Nesting level measures beyond a threshold identify **those source code units or parts that are likely to be difficult** to understand, fix, or maintain.

Halstead Complexity Metric

- ▶ Developed by Maurice Halstead of Purdue in the 1970's to analyze program source code complexity.
 - ▶ Halstead software metrics are based on static source code analysis that concentrates on the lexicographically complexity of source code.
 - ▶ Operators and operands are defined by their relationship to each other - in general, an operator carries out an action and an operand participates in such an action.
 - ▶ The underlying assumption of Halstead is that programming requires cognitive efforts (elementary cognitive steps) that can be quantitatively related to operators and operands.

Halstead Complexity Metric (2)

- ▶ The Halstead metric uses four metric units when analyzing a source program.
 - ▶ $n1$ = number of distinct operators
 - ▶ $n2$ = number of distinct operands
 - ▶ $N1$ = sum of all occurrences of $n1$
 - ▶ $N2$ = sum of all occurrences of $n2$
- ▶ In a program, there may be multiple occurrences of the same operator, such as the multiple occurrences of the “+” operator or the “IF” operator.
- ▶ The operand is either a program variable or a program constant.
- ▶ There are two more measures:
 - ▶ Program vocabulary or unique tokens, $n = n1 + n2$
 - ▶ Program length, $N = N1 + N2$

Halstead Complexity Metric (2)

- ▶ This definition of program length is quite different from the more familiar metric such as the number of lines of code.
- ▶ Four more measurements are developed from these fundamental units:
 - ▶ Volume, $V = N * (\text{Log}_2 n)$
 - ▶ Calculated program length = $n_1 \log_2 n_1 + n_2 \log_2 n_2$
 - ▶ Difficulty = $(n_1/2) * (N_2/n_2)$
 - ▶ Effort, $E = V * D$
 - ▶ Time required to program = $E/18 \text{ sec}$
 - ▶ Number of delivered bugs = $E^{(2/3)}/3000$
- ▶ Halstead metrics have been criticized for a number of reasons.
 - ▶ Halstead metrics really measure only the lexical complexity of the source program and not the structure or the logic.
 - ▶ Therefore, they have a limited value in the analysis of program complexity, let alone design characteristics.

Halstead Complexity Metric Example

```
void sort( int*a, int n ) {
    int i, j, t;

    if( n < 2 ) return;
    for( i=0 ; i < n-1; i++ ) {
        for( j=i+1 ; j < n ; j++ ) {
            if( a[i] > a[j] ) {
                t = a[i];
                a[i] = a[j];
                a[j] = t;
            }
        }
    }
}
```

3	<	3	{	1	0
5	=	3	}	2	1
1	>	1	+	1	2
1	-	2	++	6	a
2	,	2	for	8	i
9	;	2	if	7	j
4	(	1	int	3	n
4	)	1	return	3	t
6	[]				

Operators **N1 = 50 n1 = 17**
Operands **N2 = 30 n2 = 8**

Volume: $80 \log_2(24) \approx 392$

Difficulty: $D = (n1 / 2) * (N2 / n2)$
 $17/2 * 30/8 \approx 36$

Effort: $E = V * D$

Time to understand/implement (sec): $T = E/18$

793 sec $\approx$ 13 min

Bugs delivered: $E2/3/3000$

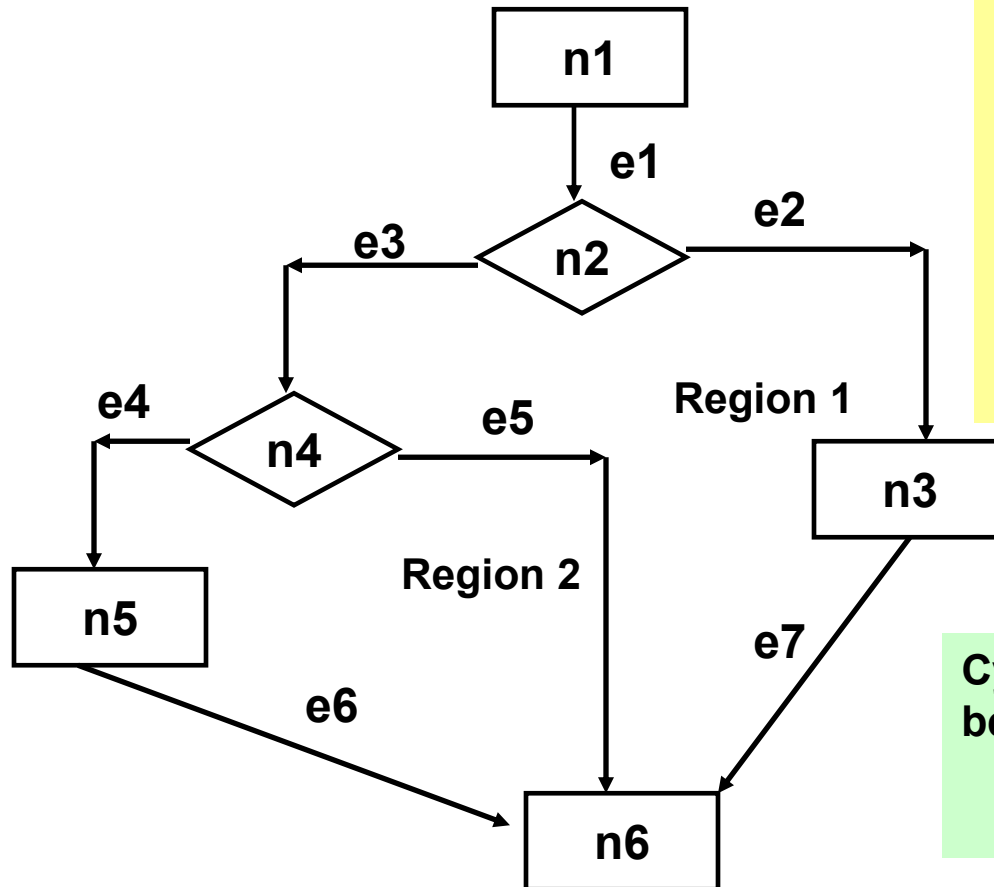
For C/C++: known to underapproximate

Running example: 0.19

McCabe's Cyclomatic Complexity

- ▶ The cyclomatic complexity measure for software resulted from T. J. McCabe's observation that
 - ▶ program quality is directly related to the complexity of the control flow or the number of branches in the source code.
- ▶ The cyclomatic complexity of a program or a detailed design is calculated from a control flow diagram representation of that design or program.
- ▶ It is represented as follows:
 - ▶ Cyclomatic complexity = $E - N + 2p$,
where E = number of edges of the graph, N = number of nodes of the graph (indivisible groups of commands of a program), and p = number of connected components (usually $p = 1$).
- ▶ McCabe's cyclomatic complexity number can also be calculated:
 - ▶ Cyclomatic complexity = number of binary decision + 1
 - ▶ Cyclomatic complexity = number of closed regions + 1

McCabe's Cyclomatic Complexity (2)



Cyclomatic complexity = $E - N + 2p$
where E = number of edges
 N = number of nodes
 p = number of connected components (usually 1)

So, for this control flow :
7 edges – 6 nodes + 2 = 3

Cyclomatic complexity number can also be computed as follows:

- number of binary decision + 1
- number of closed regions + 1

McCabe's Cyclomatic Complexity (3)

- ▶ **Definition:** Measures the structural design complexity within a program.
- ▶ **Applications:**
 - ▶ Design and code risk analysis
 - ▶ Test planning to assess the number of test cases needed to cover every decision point
- ▶ **Interpretation:**
 - ▶ Higher values → more risk and more testing required
 - ▶ 1-10 → low risk, simple
 - ▶ 50 → very high risk
- ▶ **Technical Note:**
 - ▶ The cyclomatic number represents the **maximum number of linearly independent paths** through the control flow.
 - ▶ A path is linearly independent if it contains at least one edge or segment not included in any other path.
 - ▶ Cyclomatic complexity helps determine the **number of test cases** by counting linearly independent paths in the system.

Module dependencies

- ▶ **Definition**
- ▶ A *dependency* denotes a formal relationship between two modules.
 - ▶ Dependencies can be detected by static source code analysis.
- ▶ Module *A depends* on module *B*, for example, if
 - ▶ *A* calls *B* (call dependency)
 - ▶ *A* inherits from *B* (inheritance dependency)
 - ▶ *A* uses a variable of type *B* (data/data type dependency)
- ▶ A set of modules *A1*, *A2*, ... is called a *clique* if for each pair (*Ai*, *Aj*) a dependency exists (neglecting the direction).
 - ▶ Modules of a clique are considered to be close to each other.

Henry-Kafura Information Flow

- ▶ The Henry-Kafura metric measures intermodular flow.
 - ▶ It is based on the flow of information in and out of a module
 - ▶ Parameter passing / Global variable access / Inputs/ Outputs
- ▶ An information flow is a measure of interactions between modules or between a module and its environment.
 - ▶ Fan-in: Number of information flow into a program module
 - ▶ Fan-out: Number of information flow out of a program module
 - ▶ The fan-in of a program module, mod-A, is computed as a count of modules that call mod-A. Fan-in includes the number of global data variables accessed in mod-A.
 - ▶ The fan-out of a program module, mod-A, is the number of program modules called by mod-A and the number of global data variables accessed by mod-A.
 - ▶ Fan-in of a module represents the total number of modules dependent on it. Fan-out of a module represents the number of modules this module depends on.
- ▶ The complexity for a program module p, C_p , is defined as follows:
 - ▶ $C_p = (\text{fan-in} * \text{fan-out})^2$

Henry-Kafura Information Flow -Example

Module	Mod-A	Mod-B	Mod-C	Mod-D
Fan-in	3	4	2	2
Fan-out	1	2	3	2

$$C_p \text{ for Mod-A} = (3 \times 1)^2 = 9$$

$$C_p \text{ for Mod-B} = (4 \times 2)^2 = 64$$

$$C_p \text{ for Mod-C} = (2 \times 3)^2 = 36$$

$$C_p \text{ for Mod-D} = (2 \times 2)^2 = 16$$

- ▶ The total Henry-Kafura structural complexity for all four program modules would be the sum of these, which is 125

Card and Glass (Higher Level Complexity)

- ▶ Card and Glass used the same concept of fan-in and fan-out to describe design complexity:
 - ▶ Structural complexity of module x
 - ▶ $S_x = (\text{fan-out})^2$
 - ▶ Data complexity
 - ▶ $D_x = P_x / (\text{fan-out} + 1)$, where P_x is the number of variables passed to and from the module
 - ▶ System complexity
 - ▶ $C_x = S_x + D_x$

“Good” Design Attributes

- ▶ The characteristics of a good design could be
 - ▶ Easy to understand
 - ▶ Easy to change
 - ▶ Easy to reuse
 - ▶ Easy to test
 - ▶ Easy to integrate
 - ▶ Easy to code
- ▶ According to Yourdon and Constantine (1979), these notions may be measured by two characteristics:
 - ▶ cohesion and coupling

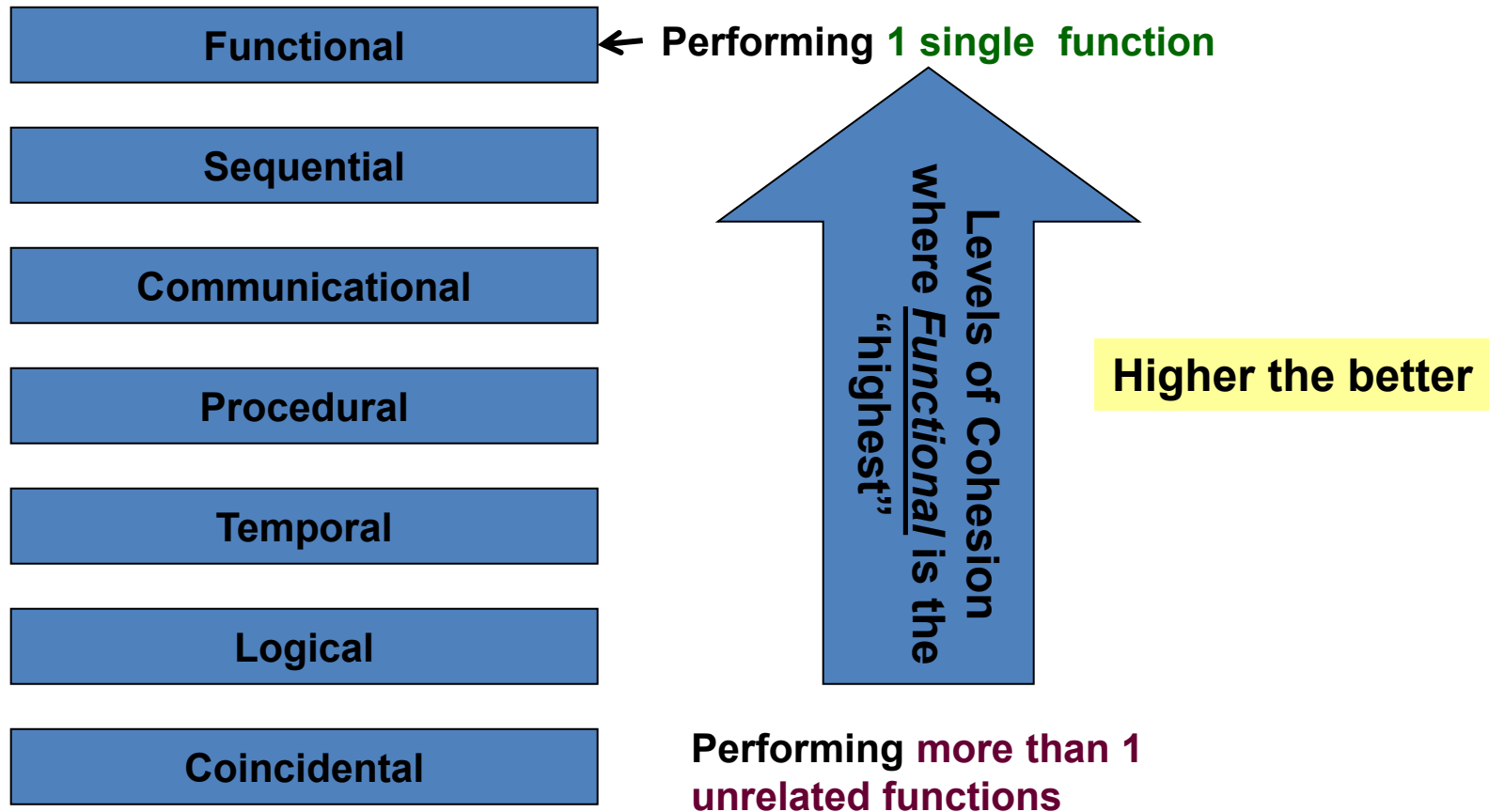
Principles of Module Dependencies

- ▶ Internal and External Dependency
 - ▶ **Coupling** of a module refers to external dependencies to other modules, that is, the degree of *inter-module dependency*.
 - ▶ **Cohesion** of a module refers to internal dependencies within the same module, that is, the degree of *intra-module dependency*.
- ▶ These two concepts are not far from the complexity measures mentioned earlier.
 - ▶ Cohesion addresses the intramodule characteristics somewhat similar to Halstead and McCabe metrics,
 - ▶ Coupling addresses the intermodule characteristics similar to the Henry-Kafura fan-in and fan-out information flow measurements and the complexity measure of Card and Glass.
- ▶ In general, we are striving for strong cohesion and loose coupling in our design.

Cohesion

- ▶ For keeping the design simple we have to ensure that each unit of the design is focused on a single purpose.
 - ▶ Cohesion: An attribute of a unit of high or detail-level design that identifies the degree to which the elements within that unit belong or are related together
- ▶ The pieces a highly cohesive design unit are all related in serving a single purpose or a single function.

Cohesion (2)



Cohesion (3)

- ▶ **Coincidental cohesion:**

- ▶ The unit of design or code performs **multiple unrelated tasks**.
- ▶ Rare in initial designs, but can appear due to **changes, bug fixes, or schedule pressures**.

- ▶ **Logical cohesion:**

- ▶ The design performs a **series of similar tasks**.
- ▶ The relationship between elements exists, but is **relatively weak**.

- ▶ **Temporal cohesion:**

- ▶ Groups elements that are **related by time** (e.g., initialization routines).

- ▶ **Procedural cohesion:**

- ▶ Elements are **procedurally related**, connected by some **control sequence**.

Cohesion (4)

- ▶ **Communicational cohesion:**

- ▶ Similar to procedural cohesion, but activities are **targeted on the same data or data sets**.
- ▶ Demonstrates **stronger internal closeness** than procedural cohesion.

- ▶ **Sequential cohesion:**

- ▶ The design unit performs **one main activity**, but the delineation of a “single” activity is **not always clear**.

- ▶ **Functional cohesion:**

- ▶ The design unit achieves **one clear goal**.
- ▶ Being **single-goal oriented**, it requires **very few changes for reuse** if the same goal is needed.
- ▶ Achieving perfect functional cohesion is **not always feasible**.

Program Slice- and Data Slice-Based Cohesion Measure

- ▶ Bieman and Ott (1994) introduced several quantitative measures of cohesion at the program level based on program and data slices.
- ▶ Consider a source program
 - ▶ A data token is any variable or constant.
 - ▶ A slice within the program or procedure is all the statements that can affect the value of some specific variable of interest.
 - ▶ A data slice is all data tokens in a slice that affect the value of a variable of interest
 - ▶ Glue tokens are the data tokens that lie in more than one data slice.
 - ▶ Superglue tokens are the data tokens that lie in every data slice.
- ▶ It is clear that glue tokens and superglue tokens are those that go across the slices and offer the binding force or strength of cohesion.
 - ▶ Weak functional cohesion = $\text{Number of glue tokens} / \text{Total number of data tokens}$
 - ▶ Strong functional cohesion = $\text{Number of superglue tokens} / \text{Total number of data tokens}$

Finding the maximum and the minimum values procedure:

```
MinMax ( z, n)
Integer end, min, max, i ;
end = n ;
max = z[0] ;
min = z[0] ;
For ( i = 0, i = < end , i++ ) {
    if z[ i ] > max then max = z[ i ] ;
    if z[ i ] < min then min = z[ i ] ;
}

return max, min;
```

<u>Data Tokens:</u>	<u>Slice max:</u>	<u>Slice min:</u>	<u>Glue Tokens:</u>	<u>Super Glue:</u>
z1	z1	z1	z1	z1
n1	n1	n1	n1	n1
end1	end1	end1	end1	end1
min1	max1	min1	i1	i1
max1	i1	i1	end2	end2
i1	end2	end2	n2	n2
end2	n2	n2	i2	i2
n2	max2	min2	03	03
max2	z2	z3	i3	i3
z2	01	02	end3	end3
01	i2	i2	i4	i4
min2	03	03	(11)	(11)
z3	i3	i3		
02	end3	end3		
i2	i4	i4		
03	z4	z6		
i3	i5	i7		
end3	max3	min3		
i4	max4	min4		
z4	z5	z7		
i5	i6	i8		
max3	max5	min5		
max4	(22)	(22)		
z5				
i6				
z6				
i7				
min3				
min4				
z7				
i8				
max5				
min5				
(33)				

A Pseudo-Code Example of Functional Cohesion Measure

Example

- ▶ For the example of finding min and max, the glue tokens are the same as the super glue tokens.
 - ▶ Super glue tokens = 11
 - ▶ Glue tokens = 11
- ▶ The data slice for min and data slice for max turns out to be the same number, 22
- ▶ The total number of data tokens is 33

The cohesion metrics for the example of min-max are:

weak functional cohesion = $11 / 33 = 1/3$

strong functional cohesion = $11 / 33 = 1/3$

If we had only computed one function (e.g. max), then :

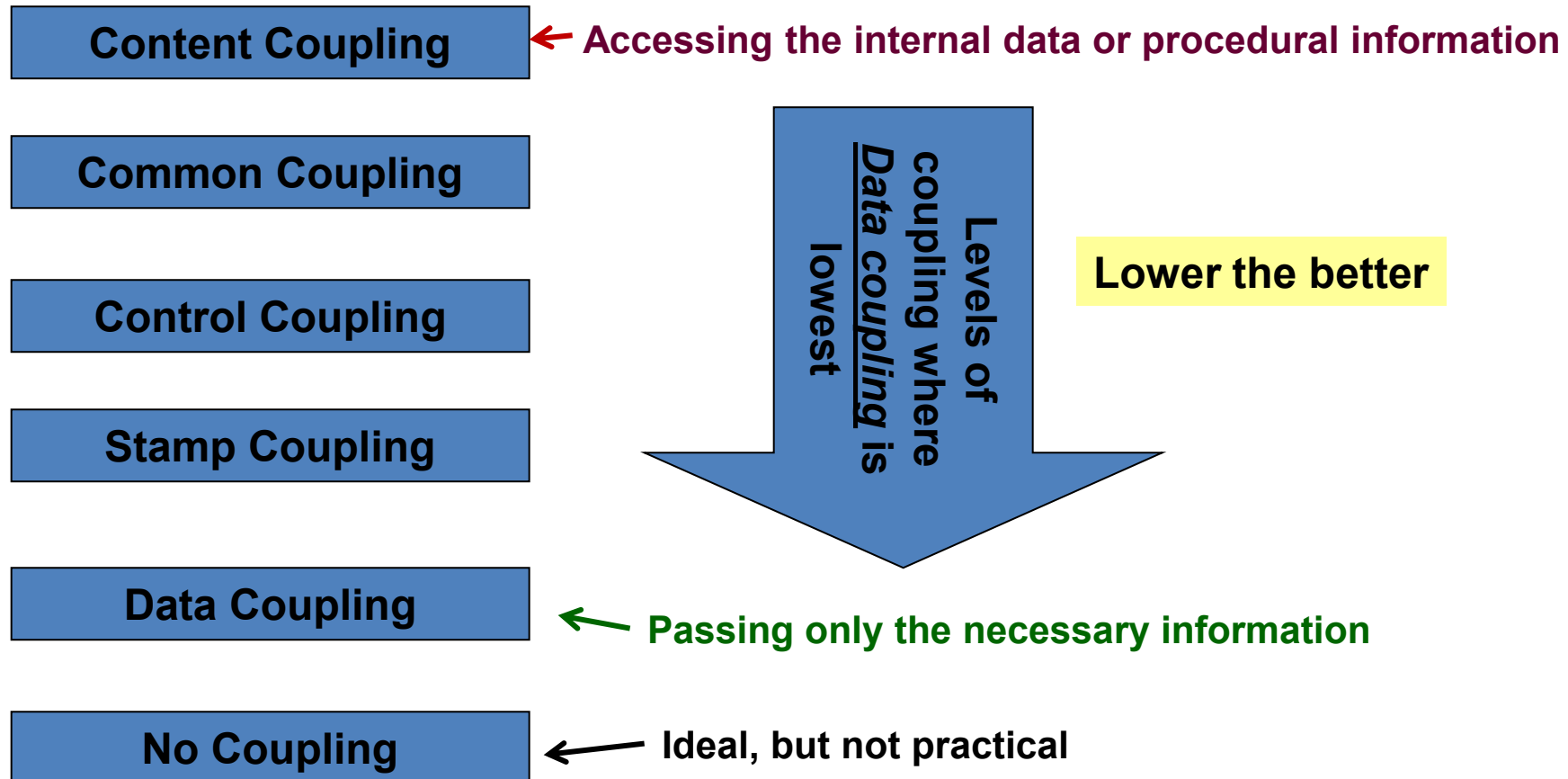
weak functional cohesion = $22 / 22 = 1$

strong functional cohesion = $22 / 22 = 1$

Coupling

- ▶ **Definition:** Degree of interaction and interdependence between software units.
- ▶ Even with **highly cohesive units**, some **coupling is inevitable**.
- ▶ **Tightly coupled classes** are difficult to:
 - ▶ Reuse in isolation
 - ▶ Understand independently
 - ▶ Modify or fix without knowing all dependent modules
- ▶ High interdependence **increases the risk** that an error in one module affects others.
- ▶ **Conclusion:** High coupling is **not desirable** in software design.

Degree of Coupling



Degree of Coupling

- ▶ **Content coupling:**
 - ▶ Two units access each other's **internal data or procedures**.
 - ▶ Any change in one unit requires a **careful review** of the other: units are almost inseparable for **reuse**.
- ▶ **Common coupling:**
 - ▶ Two units refer to the **same global variable**.
 - ▶ Used for **information exchange or control logic**. / Common in **large commercial applications**.
- ▶ **Control coupling:**
 - ▶ One unit passes **control information** to influence another unit's behavior.
 - ▶ Passed data **contains embedded control information**.
- ▶ **Stamp coupling:**
 - ▶ A unit passes a **group of data** to another unit.
 - ▶ More data than necessary may be exchanged (**less optimal than data coupling**).
- ▶ **Data coupling:**
 - ▶ Only the **necessary data** are passed between units.
 - ▶ **Low interdependence**, considered **loosely coupled**.

Fenton and Melton Measure

1. Assign each level of coupling integer from 1 through 5, respectively.
2. Evaluate software units x and y by identifying the highest level of coupling relationship between the x , y pair and assign it as i .
3. Identify all coupling relationships between x and y , and assign it as n .
4. Define the pairwise coupling as $C(x, y) = i + [n / (n + 1)]$.

► For example,

- x passes y specific data with embedded control information that will influence y ($i=2$)
- x and y share a global variable ($i=4$)
- there are two coupling relationships between x and y ($n = 2$).
- $C(x, y) = 4 + [2 / (2 + 1)]$.
- All the software units in a system are pairwise analyzed
- The overall global coupling of a system is the median value of all the pairs.

Object-Oriented Design Metrics

- ▶ How classes are interrelated to each other, and how the classes interact with each other.
 - ▶ A class is just a software entity that we want to keep simple through strong cohesion.
 - ▶ The inter-relationship and interaction among the classes should be kept simple through loose coupling.
- ▶ In OO, there are, however, some extensions to the way we would measure these attributes.
- ▶ CK Metrics (Chidamber and Kemerer)
CK metrics measure design complexity in relation to impact on external quality attributes such as maintainability, reusability, theoretically and empirically validated.
 1. Weighted methods per class (WMC)
 2. Depth of inheritance tree (DIT)
 3. Number of children (NOC)
 4. Coupling between object classes (CBO)
 5. Response for a class (RFC)
 6. Lack of cohesion in methods (LCOM)

Object-Oriented Design Metrics (2)

- ▶ **WMC (Weighted Methods per Class):**
 - ▶ Weighted sum of all methods in a class.
 - ▶ Simple approach: assign weight 1 to all methods.
- ▶ **DIT (Depth of Inheritance Tree):**
 - ▶ Maximum inheritance length from a class to its **root**.
 - ▶ Deep inheritance can make classes **hard to find and understand**.
 - ▶ Empirical results are mixed: some studies show **higher DIT → more defects**, others are inconclusive.
- ▶ **NOC (Number of Children):**
 - ▶ Number of **immediate subclasses** of a class.
 - ▶ Mixed empirical results: a large NOC may increase **system complexity**, but the effect on software quality is **uncertain**.

Object-Oriented Design Metrics (3)

- ▶ **CBO (Coupling Between Objects):**
 - ▶ Number of **classes** to which a class is coupled.
 - ▶ Similar to traditional coupling: tight coupling **increases complexity, error-proneness, and maintenance difficulty.**
- ▶ **RFC (Response For a Class):**
 - ▶ Set of **methods** involved in **response** to a message sent to an object of the class.
 - ▶ Related to coupling and **highly correlated with CBO and WMC.**
- ▶ **LCOM (Lack of Cohesion of Methods):**
 - ▶ Measures the **correlation between methods and local instance variables** of a class.

LCOM Example



CK Metrics

Lack of Cohesion in Methods

- $M = \{M_1, M_2, M_3, \dots, M_m\}$ Set of methods of a class
- $A = \{A_1, A_2, A_3, \dots, A_a\}$ Set of attributes of a class
- $\mu(A_k)$ results the number of methods in M accessing A_k

$$LCOM^* = \frac{\left(\frac{1}{a} \sum_{k=1}^a \mu(A_k)\right) - m}{1 - m}$$

- Assumption that $\mu(A_k) \geq 1$
- Properties:
 - $LCOM^* = 0$, desired situation (...all methods accessing every attribute)
 - $LCOM^* = 1$, undesired situation (...only one accessing method per attribute)

– Lionel C. Briand, John W. Daly, Jürgen Wüst: „A Unified Framework for Cohesion Measurement in Object-Oriented Systems“, 1998

User-Interface Design

- ▶ Instead of worrying about reducing software defects, we must now worry about reducing human errors.
- ▶ It is still important to understand what is it that makes the interface easier to understand, navigate, and use.
- ▶ What is user-friendliness, and what characterizes a good user-interface design?
- ▶ The important characteristic here is that the interface has more to do with people rather than software systems

Good UI Features

- ▶ Mandel's 3 “golden rules” for UI design
 - ▶ Place the user in control
 - ▶ Reduce the users' memory load (G. Miller's 7 + or - 2)
 - ▶ Consistency (earlier - design completeness and consistency)
- ▶ Shneiderman and Plaisant (8 rules for design)
 - ▶ Consistency
 - ▶ Short cuts for frequent (or experienced) users
 - ▶ Informative feedback
 - ▶ Dialogues should result in closure
 - ▶ Strive for error prevention and simple error handling
 - ▶ Easy reversal of action (“undo” of action)
 - ▶ Internal locus of control
 - ▶ Reduce short term memory

Usability Evaluation and Testing

- ▶ **Key Factors in Analyzing Application Interfaces**
- ▶ **Metrics measured:**
 - ▶ Number of users completing tasks **without help**
 - ▶ **Average time** to complete each task
 - ▶ Average number of times **help function** was used
 - ▶ Number of tasks completed within a **predefined time interval**
 - ▶ **Redo attempts** required for tasks
 - ▶ Number of times **shortcuts** were used
- ▶ **Historical approach:**
 - ▶ Early usability testing was performed **post-system** and **late in development**
 - ▶ Only **major issues** were fixed; minor issues were deferred to future releases
- ▶ **Modern approach:**
 - ▶ UI evaluation is now performed during **requirements and design phases**
 - ▶ Allows **early identification and resolution** of usability issues



References

- ▶ Frank Tsui, Orlando Karam, Barbara Bernal, Software engineering (Third Edition), Jones & Bartlett Learning
- ▶ OpenHPI - Automated Visual Software Analytics - Prof. Dr. Jürgen Döllner - 2015
- ▶ Robert C. Martin: Clean Architecture: A Craftsman's Guide to Software Structure and Design, Prentice Hall
- ▶ Rainer Koschke: Survey of Research on Software Clones. Duplication, Redundancy, and Similarity in Software 2006
- ▶ <http://butunclebob.com/ArticleS.UncleBob.PrinciplesOfOod>
- ▶ <https://stackify.com/dependency-inversion-principle/>
- ▶ <https://www.javacodegeeks.com>

MATERIALE DIDATTICO INTERNO DEL CORSO DI Ingegneria del Software.

La diffusione e pubblicazione on line del materiale è assolutamente proibita.